

ASSIGNMENT – 10.3

2303A51355

Batch-10

Task-1

Prompt: [act as ai-assisted bug detection.](#)

[#write a factorial of a num code and test it with\(5\) and identify the locigal bug in the code and also explain why that bug occurs\(eg: off-by-one error\).](#)

code :

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result *= i  
    return result  
print(factorial(5))
```

Test Output: The function returns 24 instead of the expected 120.

Logical Bug Identified:

This is an off-by-one error. The loop uses `range(1, n)` which generates numbers from 1 to $n-1$.

Why the bug occurs:

`range(1, n)` with $n=5$ produces: [1, 2, 3, 4]

The calculation becomes: $1 * 1 * 2 * 3 * 4 = 24$ (which is $4!$)

The final number 5 is never multiplied, so we get one number short.

#correct version:

```
def factorial(n):  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
    return result
```

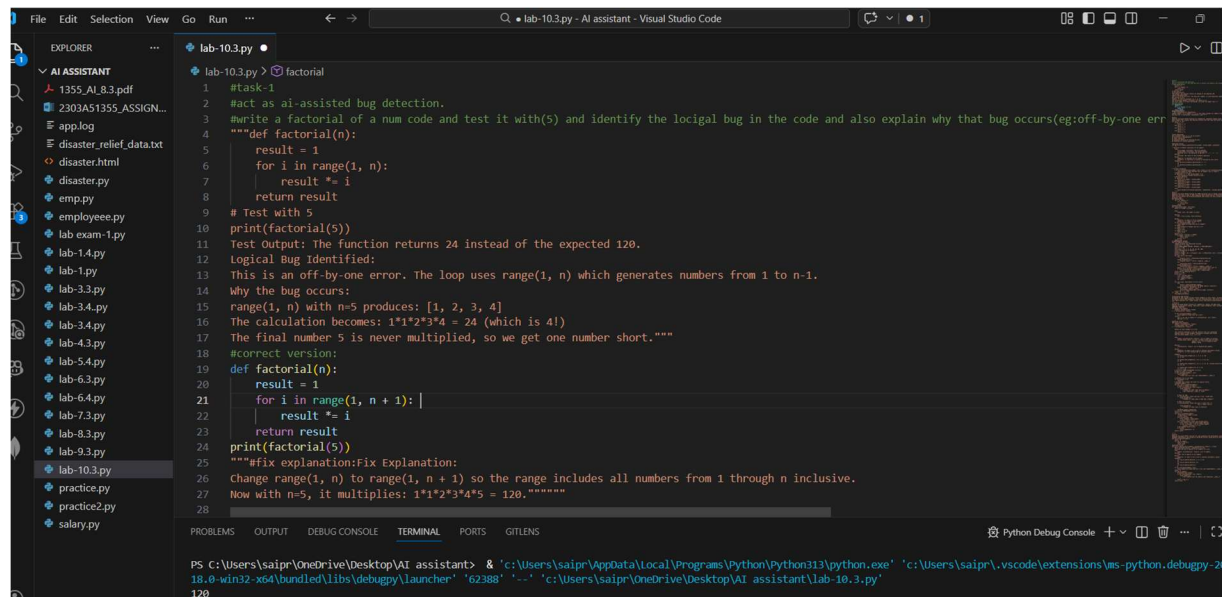
```
print(factorial(5))
```

Fix Explanation:

Change `range(1, n)` to `range(1, n + 1)` so the range includes all numbers from 1 through `n` inclusive.

Now with `n=5`, it multiplies: $1 \times 1 \times 2 \times 3 \times 4 \times 5 = 120$.

Output :



```
1 #task-1
2 #act as ai-assisted bug detection.
3 #write a factorial of a num code and test it with(5) and identify the logical bug in the code and also explain why that bug occurs(eg:off-by-one error)
4 """def factorial(n):
5     result = 1
6     for i in range(1, n):
7         result *= i
8     return result
9
10 # Test with 5
11 print(factorial(5))
12 Test Output: The function returns 24 instead of the expected 120.
13 Logical Bug Identified:
14 This is an off-by-one error. The loop uses range(1, n) which generates numbers from 1 to n-1.
15 Why the bug occurs:
16 range(1, n) with n=5 produces: [1, 2, 3, 4]
17 The calculation becomes: 1*1*2*3*4 = 24 (which is 4!)
18 The final number 5 is never multiplied, so we get one number short."""
19 #correct version:
20 def factorial(n):
21     result = 1
22     for i in range(1, n + 1):
23         result *= i
24     return result
25 print(factorial(5))
26 """#fix explanation:Fix Explanation:
27 Change range(1, n) to range(1, n + 1) so the range includes all numbers from 1 through n inclusive.
28 Now with n=5, it multiplies: 1*1*2*3*4*5 = 120."""
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Code Analysis:

- Purpose of the Function – The `factorial(n)` function calculates the factorial of a number `n`, which is the product of all positive integers from 1 to `n` (e.g., $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$).
- Logical Bug Identified – The original code used `range(1, n)`, which excludes the value `n`. Because of this, the loop only multiplies numbers from 1 to `n-1`.
- Type of Error – This is an off-by-one error, a common programming mistake where a loop runs one step less or more than required.
- Effect of the Bug – When testing with `n = 5`, the loop multiplies $1 \times 2 \times 3 \times 4$, giving 24 instead of 120, which actually computes $4!$ instead of $5!$.
- Fix Applied – Changing the loop to `range(1, n + 1)` includes the number `n` in the multiplication, ensuring the factorial is calculated correctly.

Task-2

Prompt: Analyze the given Python function for readability, parameter naming issues, and lack of documentation, then rewrite it with descriptive names, a complete docstring, input validation, and exception handling for division by zero.

#Also compare the original and improved versions and test them with valid inputs, division by zero, and invalid operations.

Code :

```
def calc(a, b, op):
```

```
    if op == '+':
```

```
        return a + b
```

```
    elif op == '-':
```

```
        return a - b
```

```
    elif op == '*':
```

```
        return a * b
```

```
    elif op == '/':
```

```
        return a / b
```

refactoried :

```
def perform_arithmetic_operation(first_number, second_number, operation):
```

```
    if not isinstance(first_number, (int, float)) or not isinstance(second_number, (int, float)):
```

```
        raise TypeError("Both operands must be numbers")
```

```
    if operation == '/' and second_number == 0:
```

```
        raise ValueError("Cannot divide by zero")
```

```
    if operation == '+':
```

```
        return first_number + second_number
```

```
    elif operation == '-':
```

```
        return first_number - second_number
```

```
    elif operation == '*':
```

```
        return first_number * second_number
```

```
    elif operation == '/':
```

```
        return first_number / second_number
```

else:

```
    raise ValueError("Invalid operation")
```

Taking input

```
a = float(input("Enter first number: "))
```

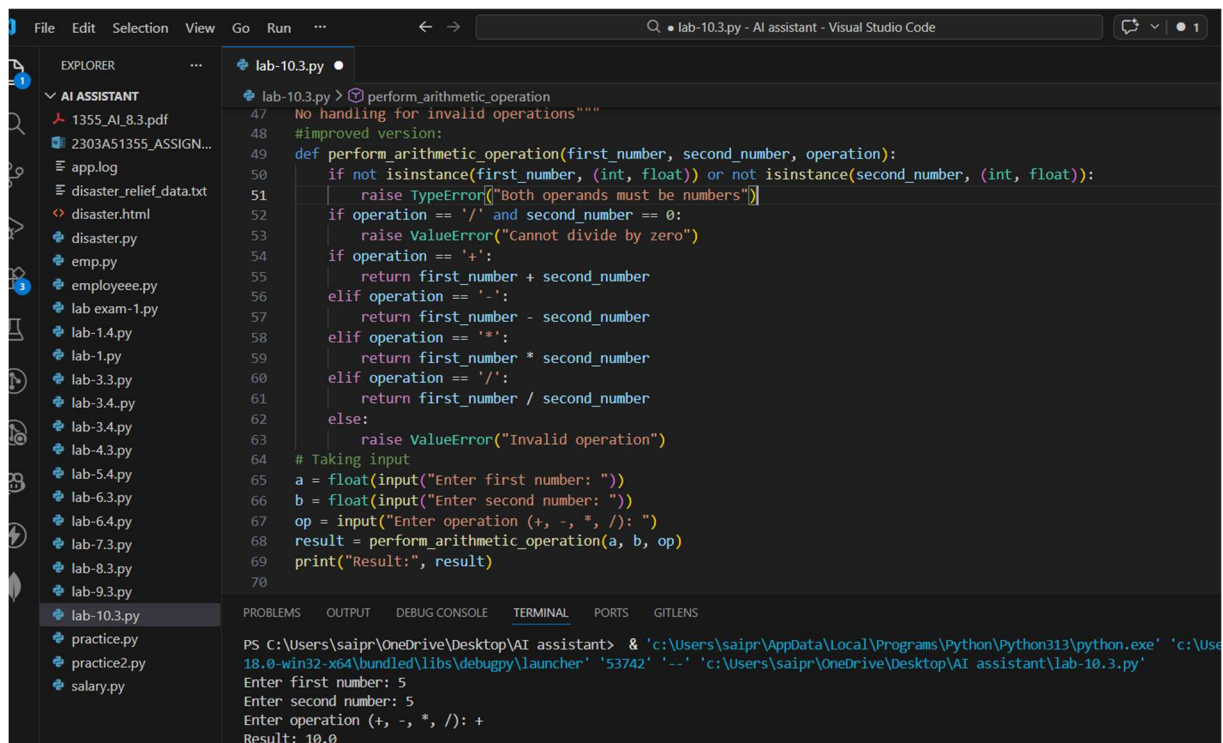
```
b = float(input("Enter second number: "))
```

```
op = input("Enter operation (+, -, *, /): ")
```

```
result = perform_arithmetic_operation(a, b, op)
```

```
print("Result:", result)
```

Output :



```
lab-10.3.py > perform_arithmetic_operation
47 No handling for invalid operations"""
48 #improved version:
49 def perform_arithmetic_operation(first_number, second_number, operation):
50     if not isinstance(first_number, (int, float)) or not isinstance(second_number, (int, float)):
51         raise TypeError("Both operands must be numbers")
52     if operation == '/' and second_number == 0:
53         raise ValueError("cannot divide by zero")
54     if operation == '+':
55         return first_number + second_number
56     elif operation == '-':
57         return first_number - second_number
58     elif operation == '*':
59         return first_number * second_number
60     elif operation == '/':
61         return first_number / second_number
62     else:
63         raise ValueError("Invalid operation")
64 # Taking input
65 a = float(input("Enter first number: "))
66 b = float(input("Enter second number: "))
67 op = input("Enter operation (+, -, *, /): ")
68 result = perform_arithmetic_operation(a, b, op)
69 print("Result:", result)
70
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\sai\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\OneDrive\Desktop\AI assistant\lab-10.3.py'
Enter first number: 5
Enter second number: 5
Enter operation (+, -, *, /): +
Result: 10.0
```

Code Analysis:

- ❑ The improved code uses clear parameter names (first_number, second_number, operation) which improves readability.
- ❑ It includes input validation to ensure the operands are numbers.

- ❑ It handles division by zero using an exception to prevent errors.
- ❑ It checks for invalid operations and raises an error if the operation is not supported.
- ❑ The program also accepts user input, allowing the function to be tested easily.

Task-3

Prompt: Analyze the given Python function for PEP8 violations such as naming conventions, spacing, indentation, and formatting.

#Refactor the code to make it PEP8 compliant while preserving its functionality and verify it with sample inputs.

#Also briefly explain how AI-based automated code reviews can help streamline code reviews in large development teams.

Code :

```
def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

refactorized:

```
def check_prime(number: int) ->
bool:
    """
    Check if a number is prime.
    """
    if not isinstance(number, int):
        raise TypeError("Input must
be an integer")
    if number < 2:
```

```
        raise ValueError("Number
must be >= 2")
    if number == 2:
        return True
    if number % 2 == 0:
        return False
    divisor = 3
    while divisor * divisor <=
number:
        if number % divisor == 0:
            return False
        divisor += 2
    return True
# Taking user input
num = int(input("Enter a number:
"))
if check_prime(num):
    print(num, "is a Prime number")
else:
    print(num, "is not a Prime
number")
```

Output:

```

lab-10.3.py X
lab-10.3.py > check_prime
82 def check_prime(number: int) -> bool:
83     """
84     Check if a number is prime.
85     """
86     if not isinstance(number, int):
87         raise TypeError("Input must be an integer")
88     if number < 2:
89         raise ValueError("Number must be >= 2")
90     if number == 2:
91         return True
92     if number % 2 == 0:
93         return False
94     divisor = 3
95     while divisor * divisor <= number:
96         if number % divisor == 0:
97             return False
98         divisor += 2
99     return True
100 # Taking user input
101 num = int(input("Enter a number: "))
102 if check_prime(num):
103     print(num, "is a Prime number")
104 else:
105     print(num, "is not a Prime number")
---
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS C:\Users\sai\r\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\r\AppData\Local\Programs\Python\Python313\python
18.0-win32-x64\bundled\libs\debugpy\launcher' '62924' '--' 'c:\Users\sai\r\OneDrive\Desktop\AI assistant\lab-10.3
Enter a number: 6
6 is not a Prime number

```

Code Analysis :

- ☐ The function name was changed from Checkprime to check_prime to follow PEP8 snake_case naming convention.
- ☐ The refactored code has proper spacing and indentation, improving readability.
- ☐ Input validation ensures the value is an integer and greater than or equal to 2.
- ☐ The algorithm checks divisibility up to number, making it more efficient.
- ☐ AI-based code reviews help detect style issues and bugs quickly, improving code quality in large teams.

Task-4

Prompt: [#Review the given Python function for readability, naming, and edge cases.](#)

[#Improve it with better naming, type hints, input validation, and a configurable multiplier, then refactor the code and briefly reflect on AI's role in code reviews.](#)

Code :

from typing import List, Union

```
def double_even_numbers(numbers: List[Union[int, float]]) ->
List[Union[int, float]]:
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")
    return [
        num * 2
        for num in numbers
        if isinstance(num, (int, float)) and num % 2 == 0
    ]
```

#refactorized:

```
from typing import List, Union
def double_even_integers(
    numbers: List[Union[int, float]],
    include_floats: bool = False
) -> List[Union[int, float]]:
    """
    Double all even integers in a list.
    """
    if not isinstance(numbers, list):
        raise TypeError(f"Input must be a list, got
{type(numbers).__name__}")
    if len(numbers) == 0:
        return []
    validated_numbers = []
    for idx, num in enumerate(numbers):
        if not isinstance(num, (int, float)):
            raise TypeError(
```

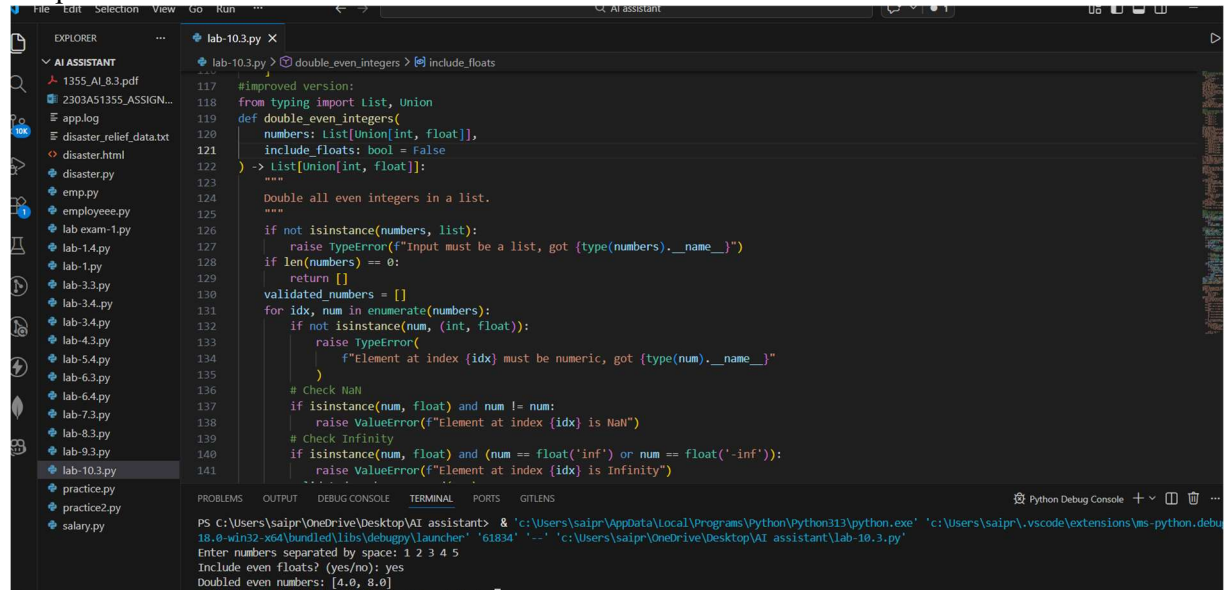
```

        f'Element at index {idx} must be numeric, got
{type(num).__name__}'
    )
    # Check NaN
    if isinstance(num, float) and num != num:
        raise ValueError(f'Element at index {idx} is NaN')
    # Check Infinity
    if isinstance(num, float) and (num == float('inf') or num ==
float('-inf')):
        raise ValueError(f'Element at index {idx} is Infinity')
    validated_numbers.append(num)
result = []
for num in validated_numbers:
    is_even = False
    if isinstance(num, int):
        is_even = num % 2 == 0
    elif isinstance(num, float) and include_floats:
        if num == int(num):
            is_even = int(num) % 2 == 0
    if is_even:
        result.append(num * 2)
return result

nums = list(map(float, input("Enter numbers separated by space:
").split()))
choice = input("Include even floats? (yes/no): ").lower()
include = True if choice == "yes" else False
result = double_even_integers(nums, include)
print("Doubled even numbers:", result)

```

Output:



```
117 #improved version:
118 from typing import List, Union
119 def double_even_integers(
120     numbers: List[Union[int, float]],
121     include_floats: bool = False
122 ) -> List[Union[int, float]]:
123     """
124     Double all even integers in a list.
125     """
126     if not isinstance(numbers, list):
127         raise TypeError(f"Input must be a list, got {type(numbers).__name__}")
128     if len(numbers) == 0:
129         return []
130     validated_numbers = []
131     for idx, num in enumerate(numbers):
132         if not isinstance(num, (int, float)):
133             raise TypeError(
134                 f"Element at index {idx} must be numeric, got {type(num).__name__}"
135             )
136         # Check NaN
137         if isinstance(num, float) and num != num:
138             raise ValueError(f"Element at index {idx} is NaN")
139         # Check Infinity
140         if isinstance(num, float) and (num == float('inf') or num == float('-inf')):
141             raise ValueError(f"Element at index {idx} is Infinity")
```

PS C:\Users\sai\r\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\r\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sai\r\.vscode\extensions\ms-python.debugpy-1.8.0-win32-x64\bundle\libs\debugpy\launcher' '61834' '-' 'c:\Users\sai\r\OneDrive\Desktop\AI assistant\lab-10.3.py'

Enter numbers separated by space: 1 2 3 4 5

Include even floats? (yes/no): yes

Doubled even numbers: [4.0, 8.0]

Code Analysis :

- Better Naming: Function name `double_even_integers` and variables are clearer and improve readability.
- Type Hints: Uses `List[Union[int, float]]` to clearly define input and output types.
- Input Validation: Checks if input is a list and ensures elements are numeric.
- Edge Cases: Handles empty lists, NaN, and Infinity values.
- AI in Code Reviews: AI helps detect errors, improve code quality, and speed up code reviews.

Task-5

Prompt: [Analyze the Python function for time complexity and performance issues when processing large datasets.](#)

[#Suggest improvements and provide an optimized version \(using built-in functions or NumPy if suitable\), then compare execution time before and after and briefly discuss readability vs performance trade-offs.](#)

Code:

```
def sum_of_squares(numbers):
```

```
    total = 0
```

```
    for num in numbers:
```

```
        total += num ** 2
```

```
    return total
```

refactorized:

```
from typing import List, Union
```

```
def sum_of_squares_basic(numbers: List[Union[int, float]]) -> float:
```

```
    """
```

```
    Basic corrected version - indentation fix only.
```

```
    Calculate the sum of squares of all numbers in a list.
```

```
    """
```

```
    if not isinstance(numbers, list):
```

```
        raise TypeError(f"Input must be a list, got {type(numbers).__name__}")
```

```
    total = 0
```

```
    for num in numbers:
```

```
        if not isinstance(num, (int, float)):
```

```
            raise TypeError(
```

```
                f"All elements must be numeric, got {type(num).__name__}"
```

```
            )
```

```
            total += num ** 2
```

```
    return total
```

```
nums = list(map(float, input("Enter numbers separated by space: ").split()))
```

```
result = sum_of_squares_basic(nums)
```

```
print("Sum of squares:", result)
```

Output :

```
163     for num in numbers:
164         total += num ** 2
165     return total"""
166 #improved version:
167 from typing import List, Union
168 def sum_of_squares_basic(numbers: List[Union[int, float]]) -> float:
169     """
170     Basic corrected version - indentation fix only.
171     calculate the sum of squares of all numbers in a list.
172     """
173     if not isinstance(numbers, list):
174         raise TypeError(f"Input must be a list, got {type(numbers).__name__}")
175     total = 0
176     for num in numbers:
177         if not isinstance(num, (int, float)):
178             raise TypeError(
179                 f"All elements must be numeric, got {type(num).__name__}"
180             )
181         total += num ** 2
182     return total
```

PS C:\Users\saipr\OneDrive\Desktop\AI assistant> & 'c:\Users\saipr\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\saipr\.vscode\extensions\ms-python.18.0-win32-x64\bundled\libs\debugpy\launcher' '63636' '--' 'c:\Users\saipr\OneDrive\Desktop\AI assistant\lab-10.3.py'

Enter numbers separated by space: 1 2 3 4 5

Sum of squares: 55.0

Code Analysis :

- ☐ Time Complexity: The function has $O(n)$ time complexity because it processes each element once.
- ☐ Performance: For large datasets, Python loops may be slower. Built-in functions or NumPy can improve speed.
- ☐ Input Validation: The improved code checks if input is a list and elements are numeric.
- ☐ Code Quality: Type hints and error handling improve readability and maintainability.
- ☐ Trade-off: Simple loops are easy to read, while optimized methods give better performance.